PROJECT SYNOPSIS &
IMPLEMENTATION PLAN

AI-Based Logic Gate Fault Detection Using Truth-Table Patterns

A machine-learning approach for identifying digital logic behaviour and detecting output faults

Student Name	_____
Register Number	_____
Department	_____
College / Institution	_____
Guide Name	_____
Academic Year	2026–2027

Prepared as a proposed academic project. Experimental results will be added after implementation and testing.

ABSTRACT

Digital logic gates are basic building blocks of digital systems. A logic gate is expected to produce a defined output for every possible input combination. In practical circuits, faults may cause an output to behave incorrectly. This project proposes a machine-learning-based approach to analyse truth-table output patterns and identify normal logic behaviour or simple output faults such as stuck-at-0 and stuck-at-1.

The project will generate truth tables for common two-input gates, create fault-injected patterns, encode each complete output pattern as a feature vector, and train classification models using Scikit-learn. Decision Tree will be used as the initial model, with Random Forest or Support Vector Machine considered for comparison. The model will be evaluated using accuracy, precision, recall, F1-score, and a confusion matrix. Logisim Evolution will be used to verify the expected logic behaviour. The work will clearly distinguish between detecting an output fault pattern and identifying the original physical gate, because different gates can produce identical faulty patterns.

Keywords: Logic gates, truth table, fault detection, stuck-at fault, machine learning, Decision Tree, classification, digital circuits.

1. INTRODUCTION

Logic gates perform Boolean operations on binary inputs (0 and 1). Common gates include AND, OR, NAND, NOR, XOR, and XNOR. Their behaviour is normally represented using truth tables. A fault in a digital circuit may cause a signal to remain at logic 0 or logic 1 regardless of the intended logic operation. These conditions are commonly described as stuck-at-0 and stuck-at-1 faults.

Traditional checking can compare expected and observed outputs. This project explores how a machine-learning classifier can learn patterns from generated truth-table data and classify the observed behaviour. The model is intended as an educational prototype for pattern-based fault analysis, not as a replacement for industrial automatic test equipment.

2. PROBLEM STATEMENT

To develop a Python-based system that analyses complete input-output truth-table patterns of logic gates and classifies them as known normal gate behaviours or simple stuck-at output-fault behaviours. The system should be tested on held-out examples and its performance should be reported using standard classification metrics.

3. OBJECTIVES

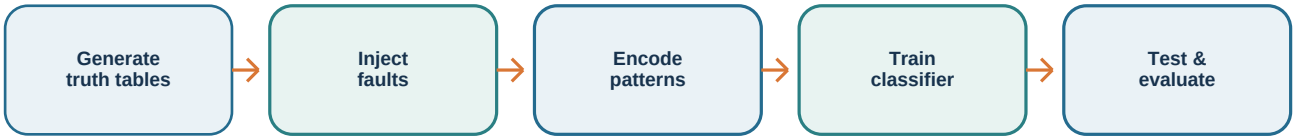
- Generate correct truth tables for AND, OR, NAND, NOR, XOR, and XNOR gates.
- Create simulated stuck-at-0 and stuck-at-1 output conditions.
- Represent a complete truth table as a fixed-length feature vector.
- Train and compare suitable classification models.
- Evaluate the model using accuracy, precision, recall, F1-score, and a confusion matrix.
- Verify logic outputs using Logisim Evolution and document limitations.

4. SCOPE OF THE PROJECT

The initial version focuses on two-input combinational gates and faults applied at the observed gate output. NOT gate behaviour can be added as a separate one-input case in an extended version. The project uses simulated data first; optional hardware testing may be added if components and laboratory access are available. It does not claim to diagnose every internal transistor-level defect or locate a fault inside a physical chip.

5. PROPOSED METHODOLOGY

The system follows the sequence shown below.



5.1 Truth-table generation

For two binary inputs A and B, the four possible combinations are 00, 01, 10, and 11. The expected output is calculated using Boolean operations. A fixed input order must be used for every gate so that the feature positions have the same meaning throughout the dataset.

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

5.2 Fault generation

For the simple output-fault model, the normal gate output is replaced by a constant value for every input combination. A stuck-at-0 output produces the pattern 0000; a stuck-at-1 output produces 1111. These patterns represent the observed behaviour, regardless of which gate was originally connected.

Input A	Input B	Normal AND	AND with output stuck-at-0	AND with output stuck-at-1
0	0	0	0	1
0	1	0	0	1
1	0	0	0	1
1	1	1	0	1

5.3 Feature encoding and labels

Each complete truth table is encoded as a four-element vector in the fixed input order [00, 01, 10, 11]. For example, a normal AND gate is [0, 0, 0, 1], while a stuck-at-0 output is [0, 0, 0, 0]. The target labels for the first version are AND_Normal, OR_Normal, NAND_Normal, NOR_Normal, XOR_Normal, XNOR_Normal, Stuck_at_0, and Stuck_at_1. This produces an eight-class behaviour classifier.

Important identifiability note: once the output is forced to 0 or 1, the observed truth table no longer contains enough information to determine the original gate. Therefore, the fault classes should be labelled as Stuck_at_0 and Stuck_at_1 without claiming the original gate identity. If the original gate is known from the circuit design, that information can be stored separately.

5.4 Dataset design and train/test strategy

A truth-table behaviour is one dataset sample; its four output bits are the input features and its behaviour category is the target. The dataset must be split by complete samples, not by individual truth-table rows. For a deterministic basic dataset there are only a small number of unique patterns, so a machine-learning score alone may be misleading. The project should report the exact number of unique patterns and use additional test cases such as repeated measurements, controlled observation noise, or partial-input observations only if these are clearly defined and kept separate between training and testing. The clean deterministic baseline should always be reported.

5.5 Model training

Decision Tree is the recommended starting model because it is simple to interpret. Random Forest may be tested as an ensemble alternative, and Support Vector Machine (SVM) may be included for comparison. Scikit-learn will be used to train each model. Model selection should be based on held-out evaluation, not only on training accuracy.

6. SOFTWARE AND TOOLS REQUIRED

Tool	Purpose
Python	Main programming language for data generation, model training, and testing.
NumPy	Binary arrays and numerical operations.
Pandas	Creating, cleaning, and exporting datasets as CSV files.
Scikit-learn	Classification models, train/test utilities, and evaluation metrics.
Google Colab / Jupyter	Environment for writing and running Python notebooks.
Logisim Evolution	Logic-gate simulation and truth-table verification.
Optional hardware	Digital ICs, breadboard, wires, and a regulated logic supply for physical demonstration under supervision.

7. IMPLEMENTATION PLAN

- **Step 1:** Set up Python and import NumPy, Pandas, and Scikit-learn.
- **Step 2:** Generate the normal truth tables for the selected gates.
- **Step 3:** Generate stuck-at-0 and stuck-at-1 output patterns.
- **Step 4:** Convert each complete output table into a four-bit feature vector and assign a behaviour label.
- **Step 5:** Save the dataset as a CSV file and check for duplicate patterns and label conflicts.
- **Step 6:** Train a Decision Tree classifier; optionally compare Random Forest and SVM.
- **Step 7:** Test on held-out complete patterns or carefully designed extended test cases.
- **Step 8:** Calculate evaluation metrics and plot a confusion matrix.
- **Step 9:** Verify the normal truth tables in Logisim Evolution and prepare the final demonstration.

8. MODEL EVALUATION

The following measures will be used to assess the classifier:

- **Accuracy:** Proportion of all predictions that are correct.
- **Precision:** Of the samples predicted as a class, the proportion that truly belongs to that class.
- **Recall:** Of the actual samples in a class, the proportion correctly detected.
- **F1-score:** Harmonic mean of precision and recall.
- **Confusion matrix:** Table showing correct and incorrect predictions for each class.

No numerical result is stated in this proposal because the model has not yet been trained and tested. Actual metric values, test-set size, class-wise scores, and the confusion matrix must be inserted after implementation.

9. EXPECTED OUTCOMES

- A program that generates normal and fault-injected truth-table patterns.
- A labelled dataset in CSV format.
- A trained classification model that predicts the observed behaviour category.
- An evaluation report containing the required performance metrics.
- A Logisim-based verification record and a simple prediction demonstration.

10. LIMITATIONS AND RISK CONTROL

- A complete truth table contains only four observations for a two-input gate; the number of distinct deterministic patterns is small.
- Different faulty circuits can produce the same output pattern. The model cannot infer hidden circuit details from outputs alone.
- A stuck-at fault at the output is a simplified fault model; internal line faults may require different test patterns.
- Synthetic data may not represent electrical noise, timing faults, loading effects, or real hardware defects.
- To avoid misleading results, split data by complete pattern or independent circuit instance, and disclose any synthetic noise or augmentation.

11. PROPOSED WORK SCHEDULE

Week	Planned activity	Deliverable
1	Study logic gates and prepare truth tables.	Verified truth tables
2	Write Python code for normal and faulty patterns.	Generated dataset
3	Encode features, label classes, and inspect duplicates.	Clean CSV dataset
4	Train Decision Tree and comparison models.	Trained models
5	Test, calculate metrics, and analyse errors.	Evaluation report
6	Verify in Logisim and prepare final presentation.	Demonstration and report

12. CONCLUSION

This project proposes a structured method for analysing logic-gate truth-table patterns using machine learning. It combines Boolean logic, dataset generation, feature encoding, classification, and model evaluation. By beginning with simulated output-level stuck-at faults and verifying expected behaviour in Logisim Evolution, the project provides a manageable foundation for an engineering prototype. The final report will include measured results only after the implementation and independent testing are completed.

13. FUTURE ENHANCEMENTS

- Extend the dataset to NOT gates and multi-input logic gates.
- Study internal stuck-at faults on selected circuit lines rather than only the output.
- Add timing-related or intermittent fault simulations where suitable test data is available.
- Connect a physical logic-gate circuit and capture its outputs for comparison with simulated data.
- Develop a simple user interface that accepts four output bits and displays the predicted behaviour and confidence information where supported.

14. REFERENCES

1. Scikit-learn Developers, “Decision Trees,” Scikit-learn User Guide. https://scikit-learn.org/stable/modules/tree.html
2. Scikit-learn Developers, “Classification Metrics,” Scikit-learn User Guide. https://scikit-learn.org/stable/modules/model_evaluation.html
3. Logisim Evolution, open-source digital logic simulator. https://github.com/logisim-evolution/logisim-evolution
4. Python Software Foundation, “Python Documentation.” https://docs.python.org/3/
5. Wes McKinney, “Data Structures for Statistical Computing in Python,” Proceedings of the 9th Python in Science Conference, 2010.

Submission note
This document is a project proposal and implementation plan. Replace the student/institution fields on the cover page and update the report with actual code outputs, screenshots, test details, and measured evaluation results before submitting it as a completed-project report.